

EPL and Esper

Streaming joins and more

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml

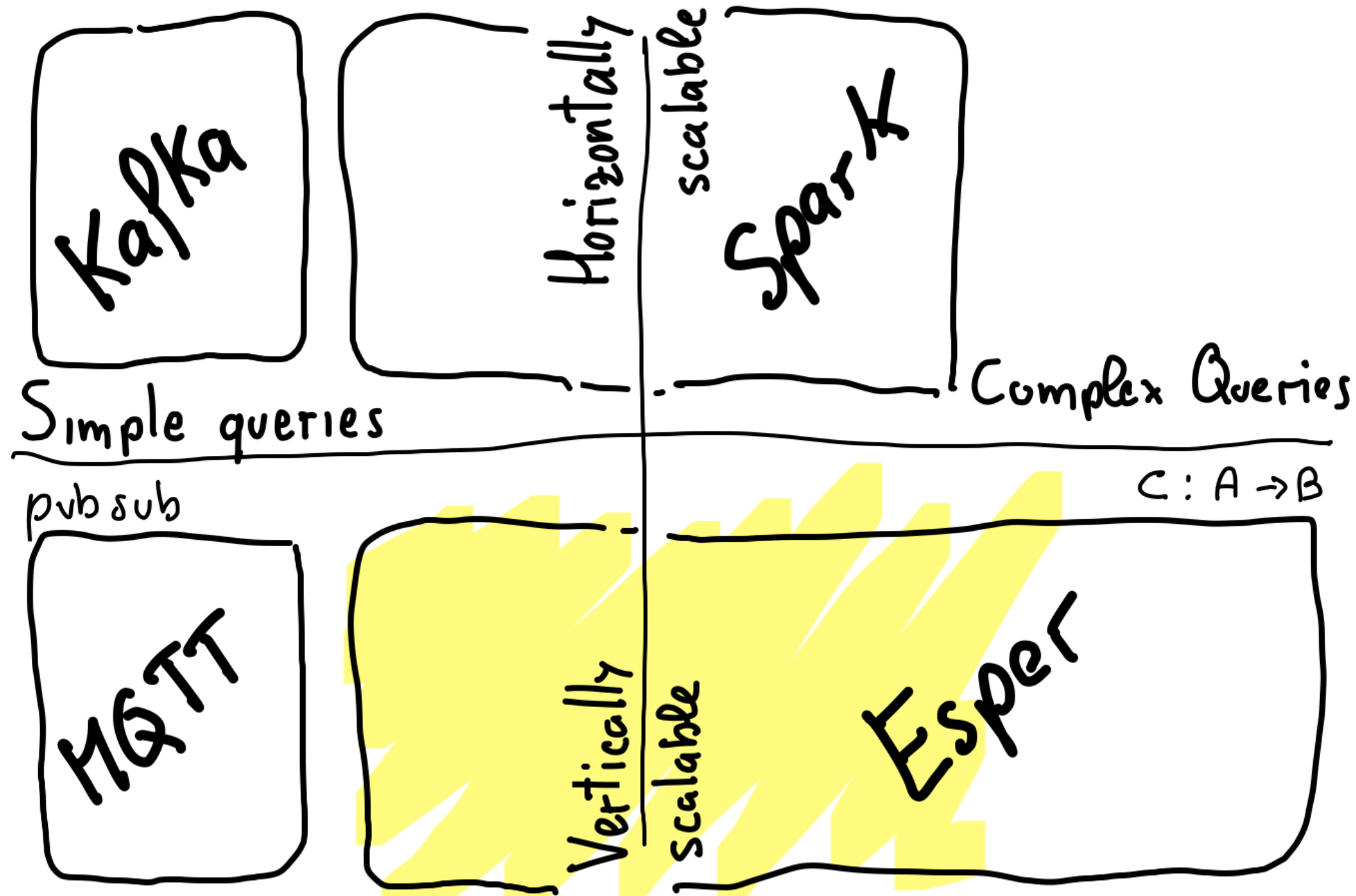


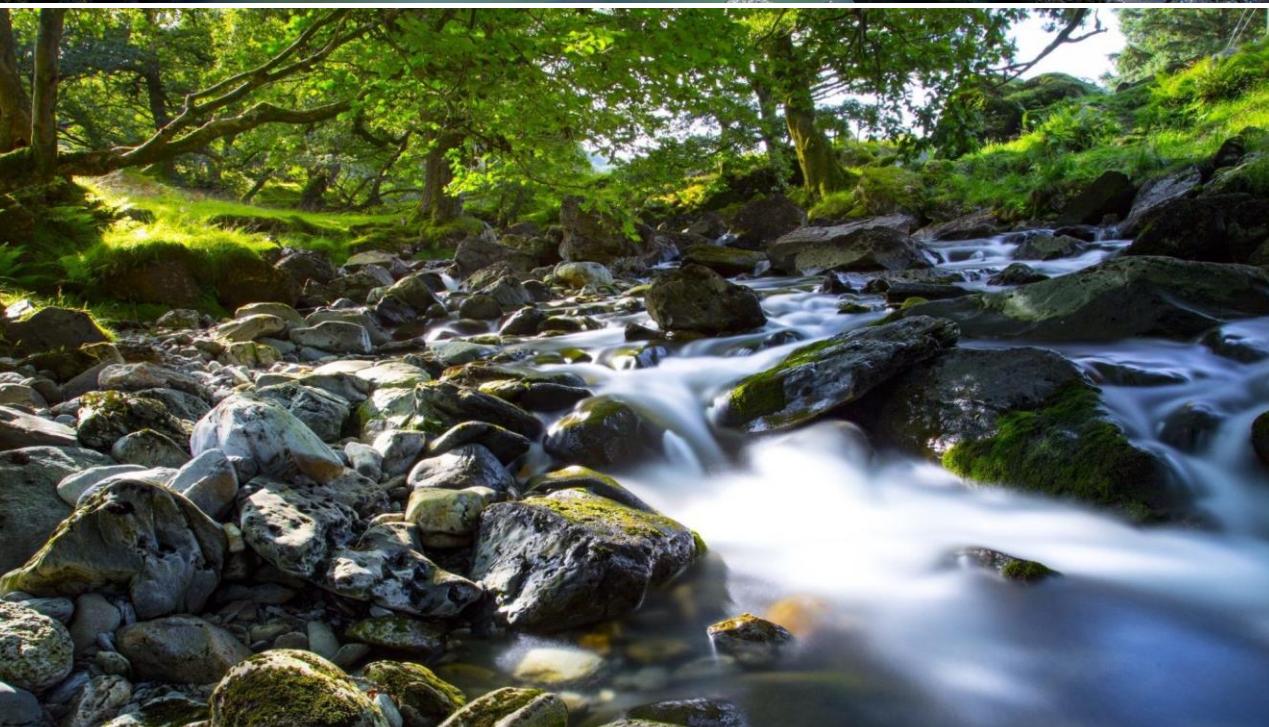
POLITECNICO
MILANO 1863



POLITECNICO
MILANO 1863

Positioning this lecture

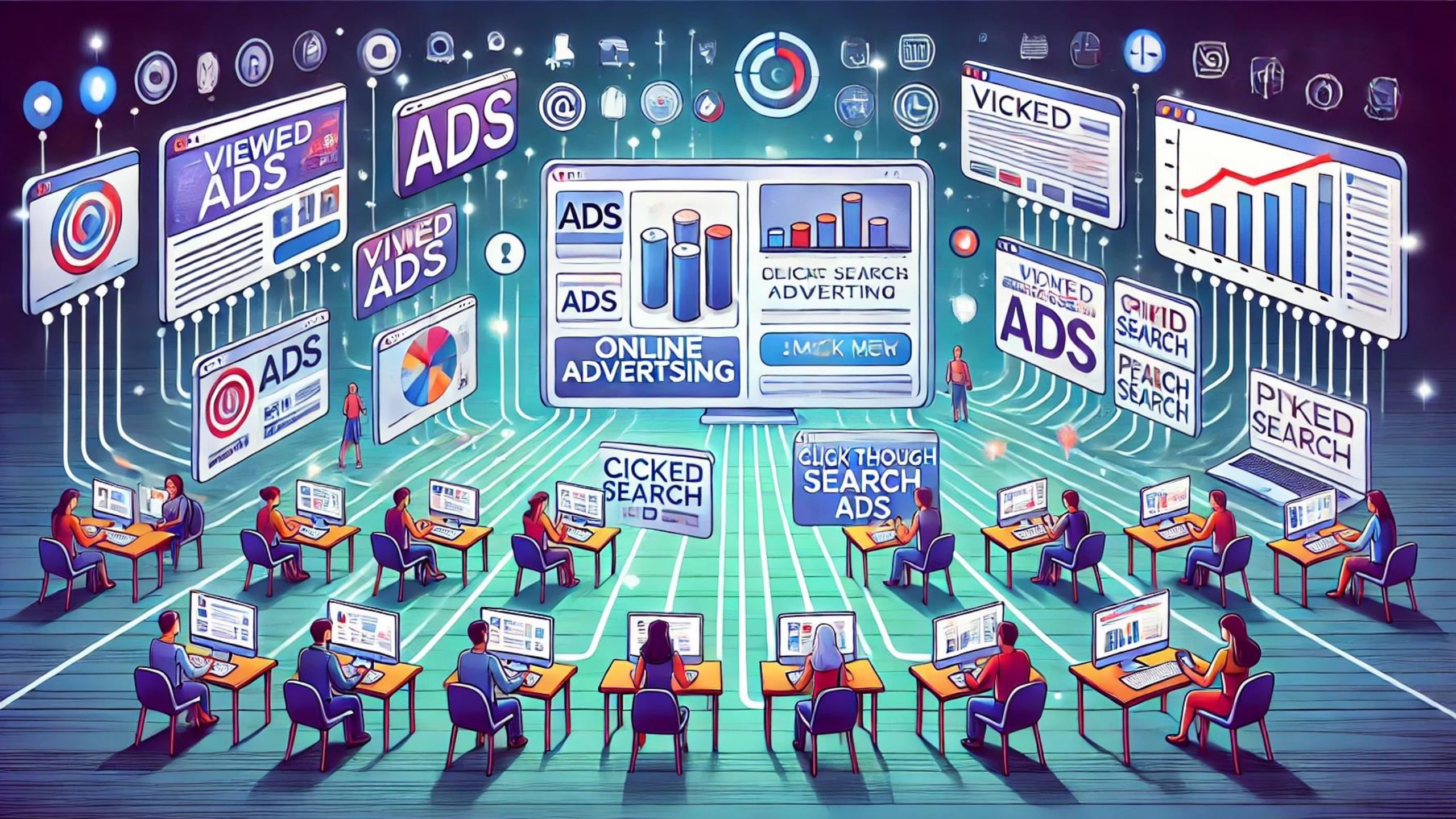




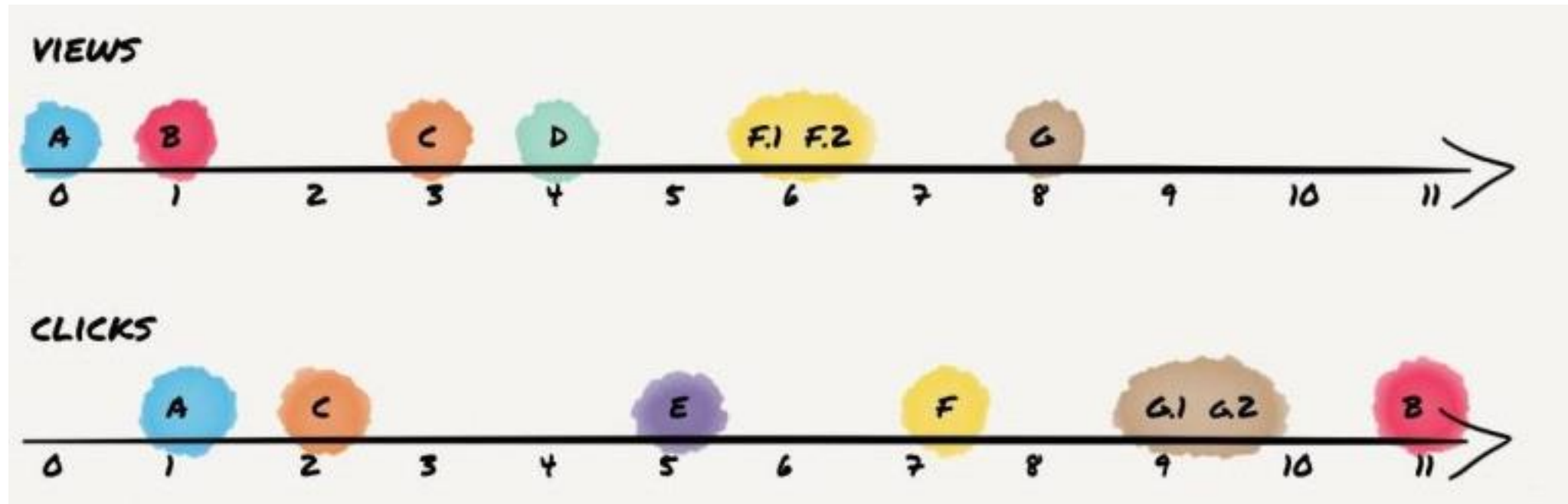


POLITECNICO
MILANO 1863

The Running Example



The two streams





The two streams

7 Scenarios

- a click event arrives 1 sec after the view
- a click event arrives 10 sec after the view
- a view event arrives 1 sec after the click
- there is a view event but no click
- there is a click event but no view
- there are two consecutive view events and one click event 1 sec after the first view
- there is a view event followed by two click events shortly after each other



SQL joins

- In SQL, joins are performed on tables, which are static data structures. The semantics of SQL joins are based on a relational data model and queries that work with complete data sets.
 - Inner Join: Returns only rows with matches on both tables.
 - Left/Right Outer Join: Returns all rows from one table (left or right), with matches found in the other table, and NULL for rows with no matches.
 - Full Outer Join: Returns all rows when a match occurs in one of the tables.
- These joins operate on “static” data and usually do not consider the time dimension (unless specific time conditions are applied, but this is not intrinsic to the joins' semantics).

Code available on github!

- All the code described in this lecture is available at

<https://github.com/Streaming-Data-Analytics/Courseware/tree/main/Streaming%20Data%20Engineering/EPL/epl-join-semantics>





POLITECNICO
MILANO 1863

Stream to Stream Joins



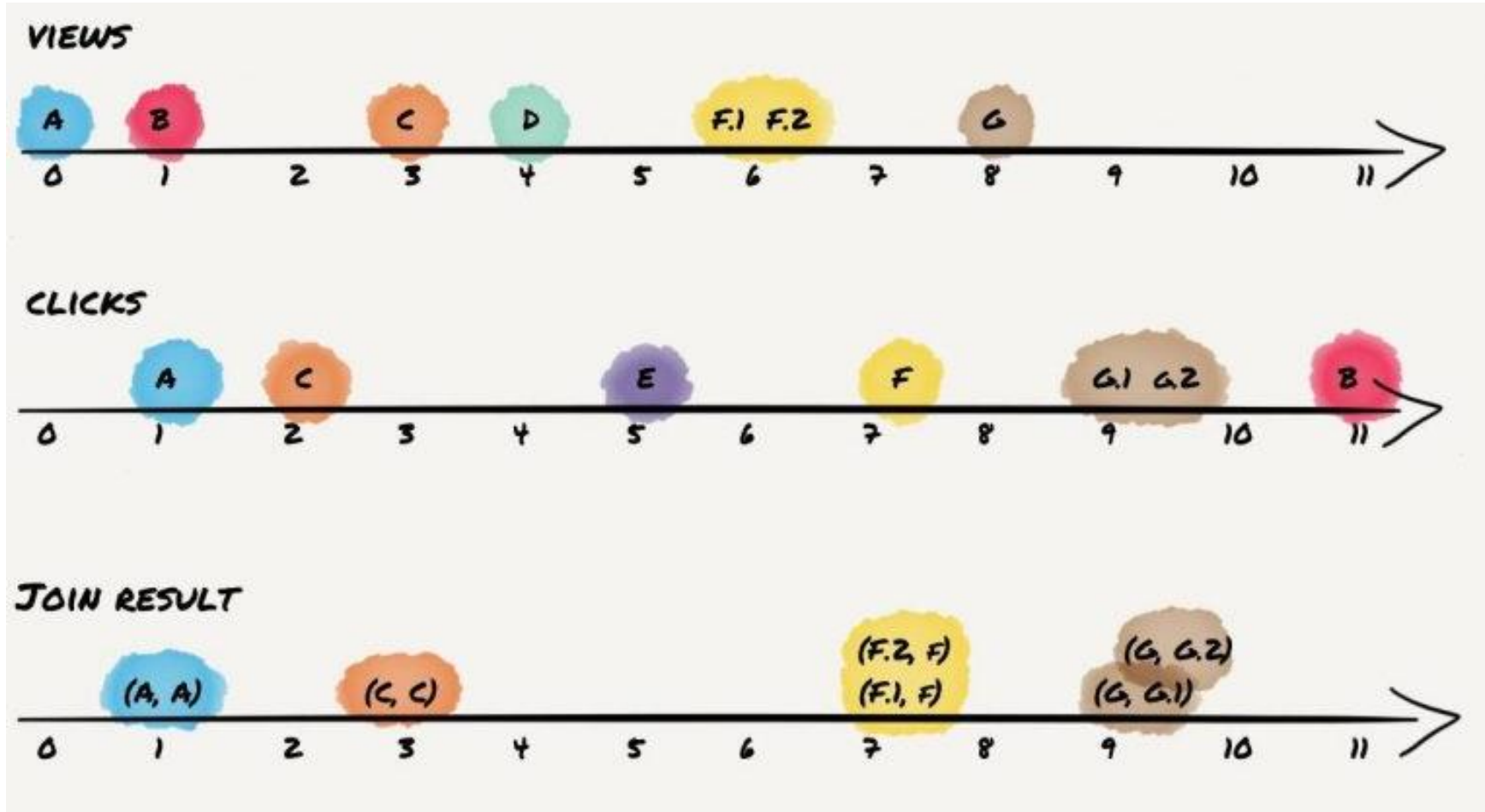
Stream to Stream Inner Join

- Joins require that at least a window is specified for each stream. In the following examples, we use windows of 9 seconds.
- Query

```
select * .win:  
from View#time(9 sec) as v  
    inner join  
    Click#time(9 sec) as c  
on v.id = c.id;
```



Stream to Stream Inner Join





Stream to Stream Inner Join

- Records A and C appear as expected, as the key appears in both streams within 9 seconds, even though they come in different order.
- Records B produce no result: although both records have matching keys, they do not appear within the time window.
- Records D and E don't join because neither has a matching key contained in both streams.
- Records F and G appear two times as the keys appear twice in the view stream for F and in the clickstream for scenario G.



Stream to Stream

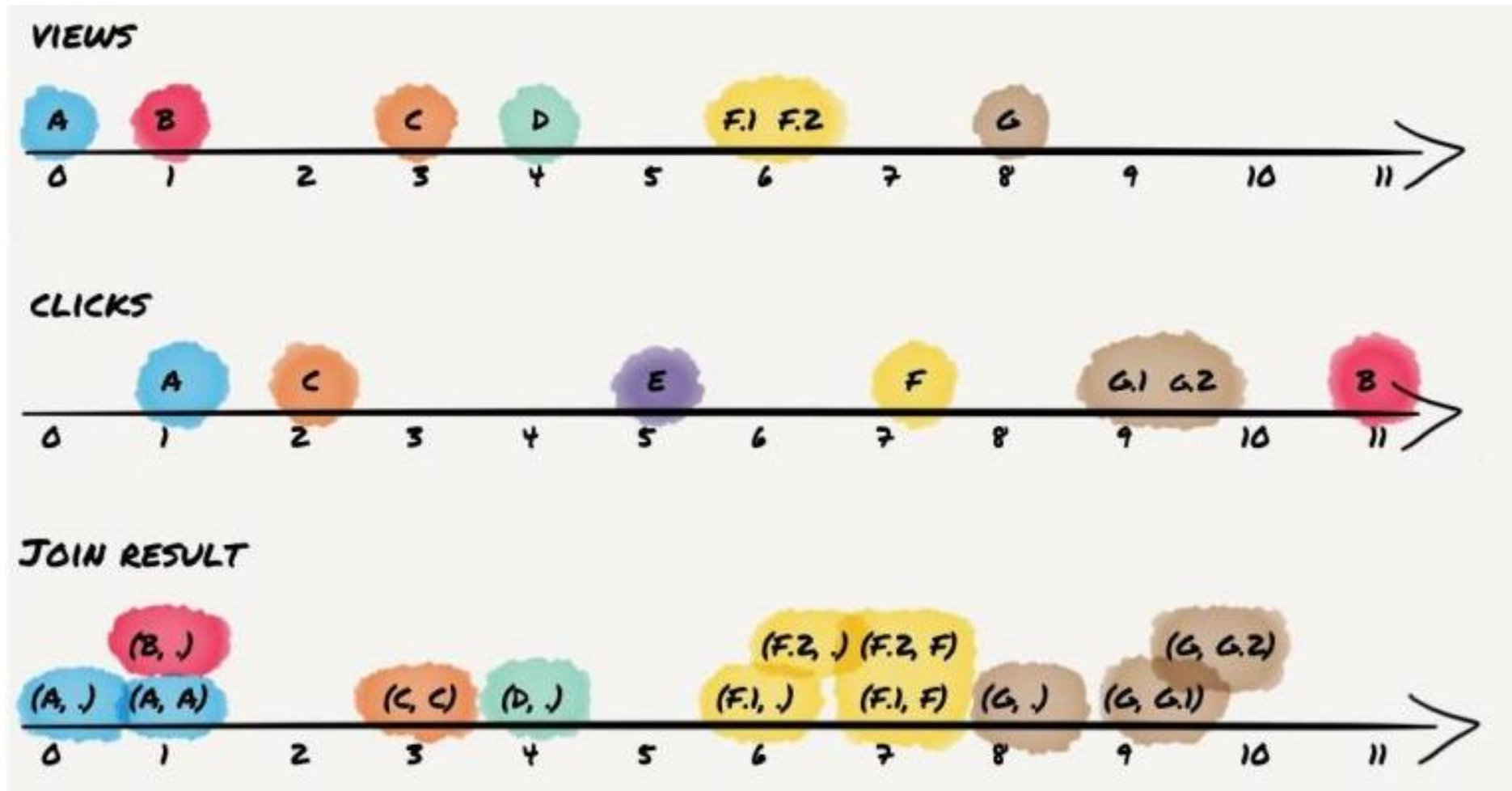
Left Outer Join

- The left join starts a computation each time an event arrives for either the left or right input stream.
- For input records of the left stream, an output event is generated every time an event arrives. If an event with the same key has previously arrived in the right stream, it is joined with the one in the primary stream. Otherwise, it is set to null.
- On the other hand, each time an event arrives in the right stream, it is only joined if an event with the same key previously arrived in the primary stream.
- Query

```
select *  
from View#time(9 sec) as v  
    left outer join  
    Click#time(9 sec) as c  
on v.id = c.id;
```



Stream to Stream Left Outer Join





Stream to Stream Left Outer Join

- The result contains all records from the inner join.
- Additionally, it contains a result record for B and D and thus contains all records from the primary (left) “view” stream.
- Also, note the results for “view” records A, F.1/F.2, and G with null (indicated as “dot”) on the right-hand side. *Those records would not be included in a SQL join.*
- As Esper provides stream join semantics and processes each record when it arrives, the right-hand window does not contain a corresponding key for primary “view” input events A, F1./F.2, and G in the secondary “click” input stream in our example and thus correctly includes those events in the result.



Stream to Stream

Full Outer Join

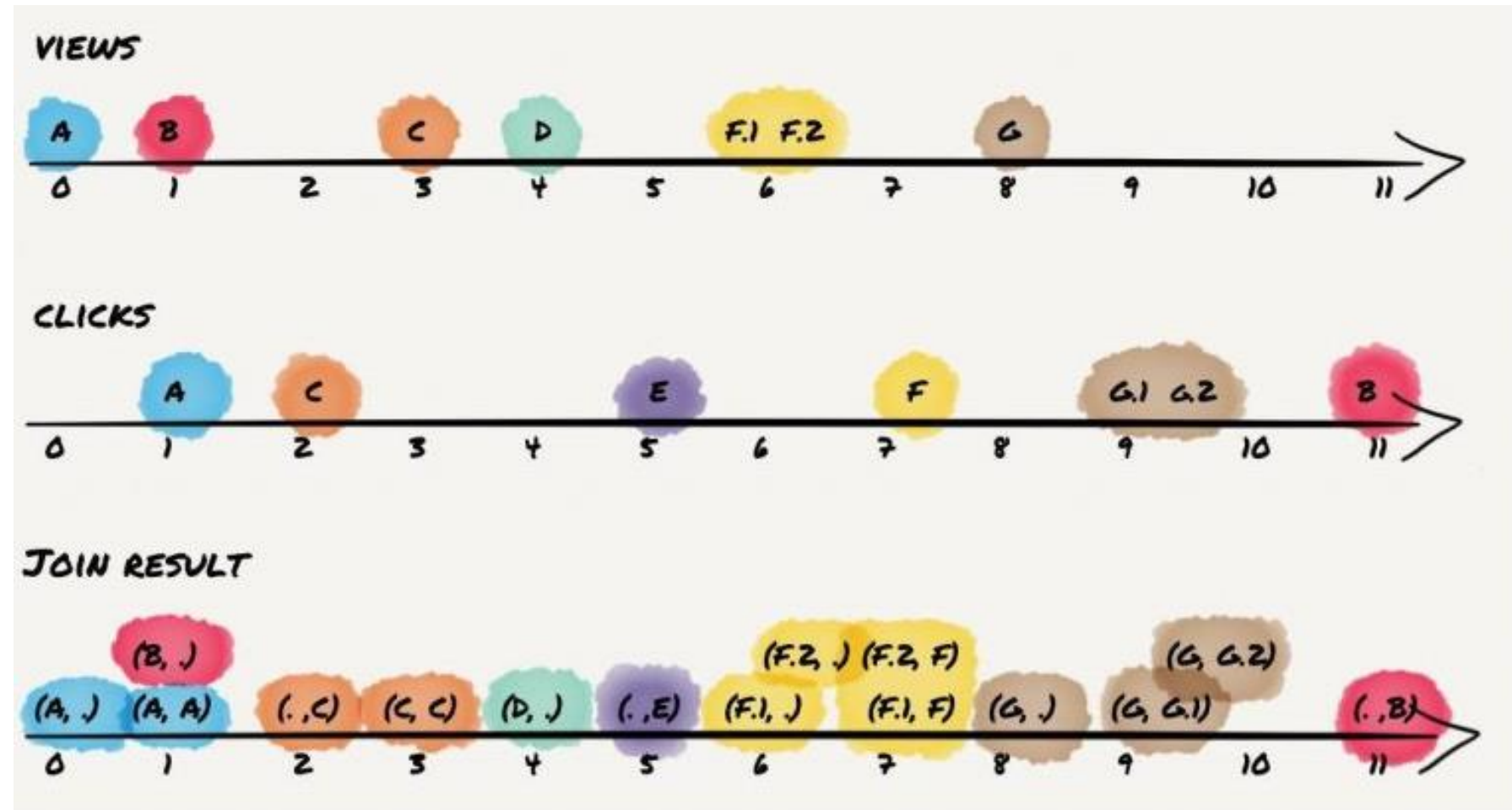
- An outer join will emit an output each time an event is processed in either stream.
- The join method will be applied to both elements if the window state already contains an element with the same key in the other stream.
- If not, it will only apply to the incoming element.

- Query

```
select *  
from View#time(9 sec) as v  
    full outer join  
    Click#time(9 sec) as c  
on v.id = c.id;
```



Stream to Stream Full Outer Join





Stream to Stream

Full Outer Join

- For record A, an event is emitted once the view is processed. There is no click yet. When the click arrives, the joined event on view and click is emitted.
- For record B, we also get two output events. However, since the events do not occur within the window, neither of these events contains both view and click (i.e., both are independent outer-join results).
- “View” record D appears in the output without a click, and the equivalent (but “reverse”) output is emitted for “click” record E.
- Records F produce 4 output events as there are two views that are emitted immediately and once again when they are joined against a click.
- In contrast, records G produce only 3 events as both clicks can be immediately joined against a view that arrived earlier.



Stream to Stream Discussion

- Joins among streams have profoundly different semantics because they are designed to run on real-time data streams, essentially unbounded record sequences.
- Events are joined only if they arrive within a specified time window. This temporal aspect is central and intrinsic to the join semantics of streams.
 - Streaming Inner Joins: Returns combined records only when a match exists within the time window.
 - Left Join: Returns all records in the left stream with matches in the right stream (where there are no matches, they are filled with null).
 - Outer Join: Similar to SQL joins, includes all records from both streams, with null for fields with no matches.
- The concept of time is crucial. Joins depend on the time synchronization of events in streams, which introduces a significant difference from traditional SQL joins.



POLITECNICO
MILANO 1863

Table to Table Joins



“Tables” in EPL

- EPL offers three ways to build tables out of unbounded streams:
 - The `keep all` window is a data window that retains all arriving events. However, care should be taken to remove events from the keep-all data window in a timely manner.
 - The `unique` window is a window that includes only the most recent among events having the same value(s) for the result of the specified expression or list of expressions.
 - The `create Table` statement with the corresponding `insert into` aggregation query since they are holders for aggregation state
- For this running example, we need the `unique` window



Table to Table Inner Join

- Joins require that at least a window is specified for each stream. In the following examples, we use unique windows to turn the stream into a "table".
- Query

```
select *  
from View#unique(id) as v  
    inner join  
    Click#unique(id) as c  
on v.id = c.id;
```


Table to Table Inner Join



POLITECNICO
MILANO 1863

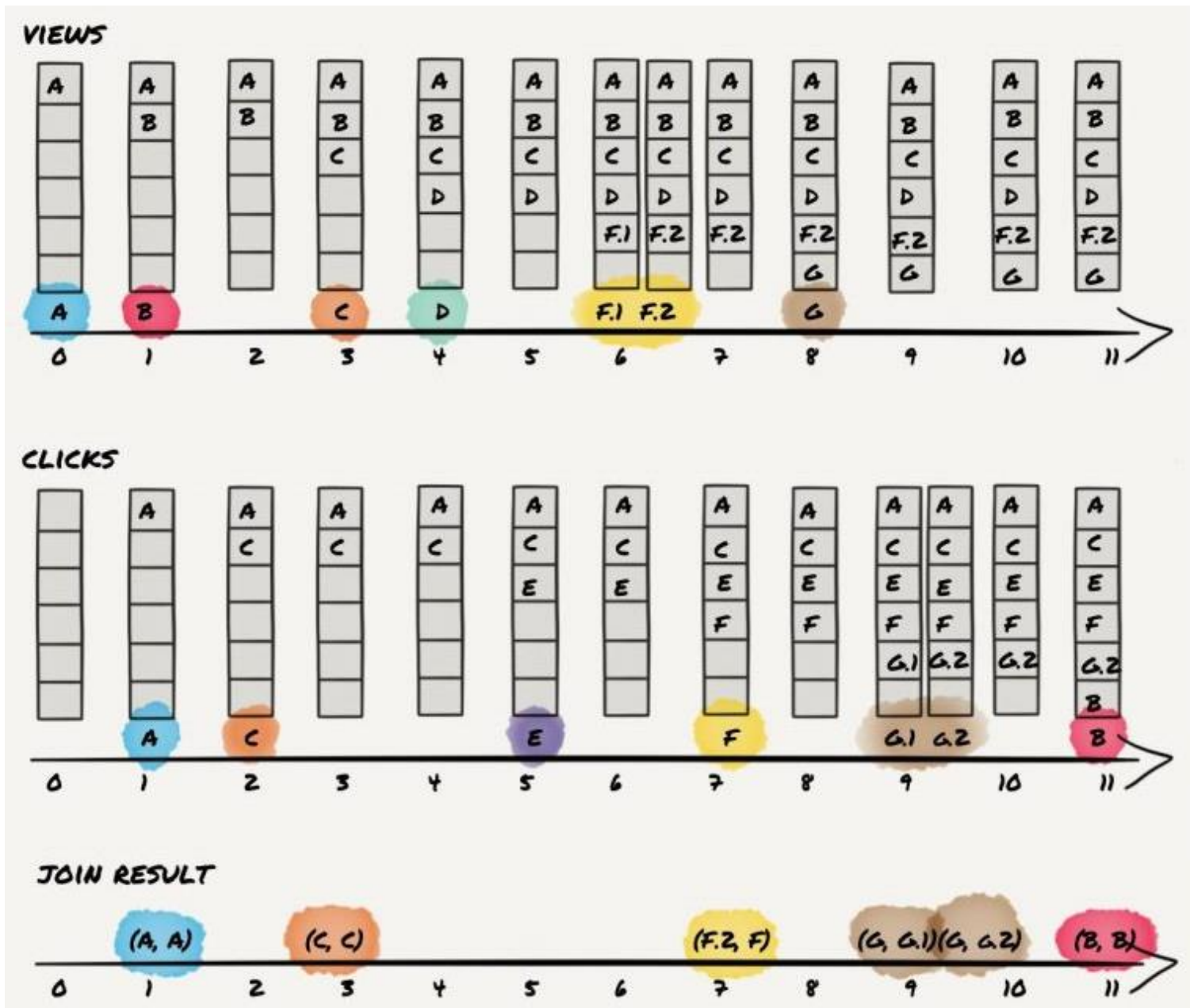




Table to Table Inner Join

- All the inner join pairs are emitted as expected. Since we're no longer windowed, even record B/B is in the result.
- Note that the result contains only one result for F but two for G. Because:
 - click F appears after views F.1 and F.2, F.2 did replace F.1 before F triggers the join computation.
 - For G, the view arrives before both clicks; thus, G.1 and G.2 join with G.
- This scenario demonstrates the update behavior of a join between two "Tables". After G.1 arrives, the join result is G.1/G. Then the click event G.2 updates the click table and triggers a recomputation of the join result to G.2/G.1



Table to Table

Left Outer Join

- Query

```
select *  
from View#unique(id) as v  
      left outer join  
      Click#unique(id) as c  
on v.id = c.id;
```

Table to Table Left Join



POLITECNICO
MILANO 1863

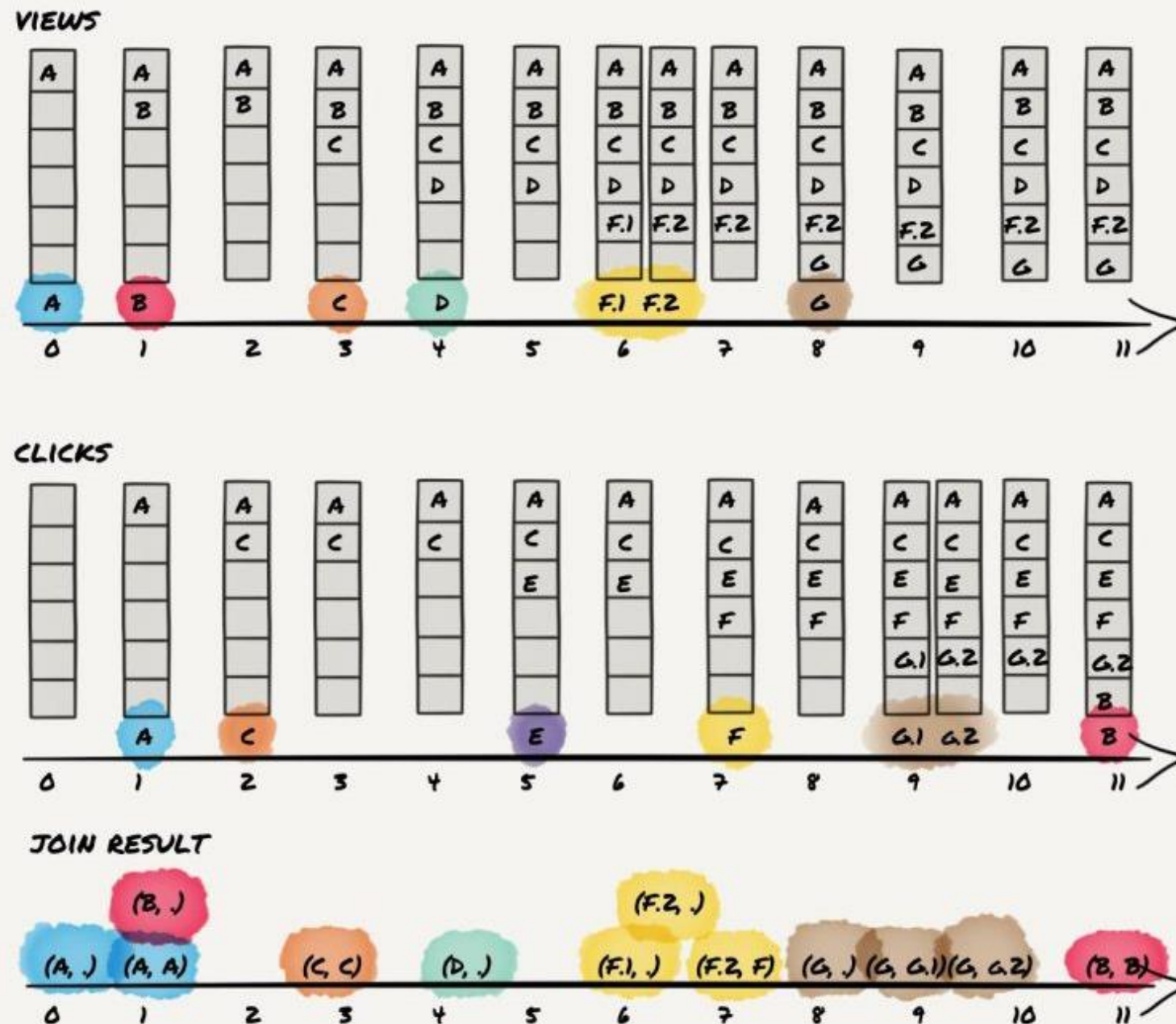




Table to Table Left Outer Join

- The result is the same as with the inner join with the addition of proper data for (left) view events A, B, D, F.1, F.2, and G that do a left join with empty right-hand side when they are processed first.
- Thus, view D is preserved and only click E is not contained as there is no corresponding view.



Table to Table

Full Outer Join

- Query

```
select *  
from View#unique(id) as v  
      full outer join  
      Click#unique(id) as c  
on v.id = c.id;
```

Table to Table Outer Join



POLITECNICO
MILANO 1863

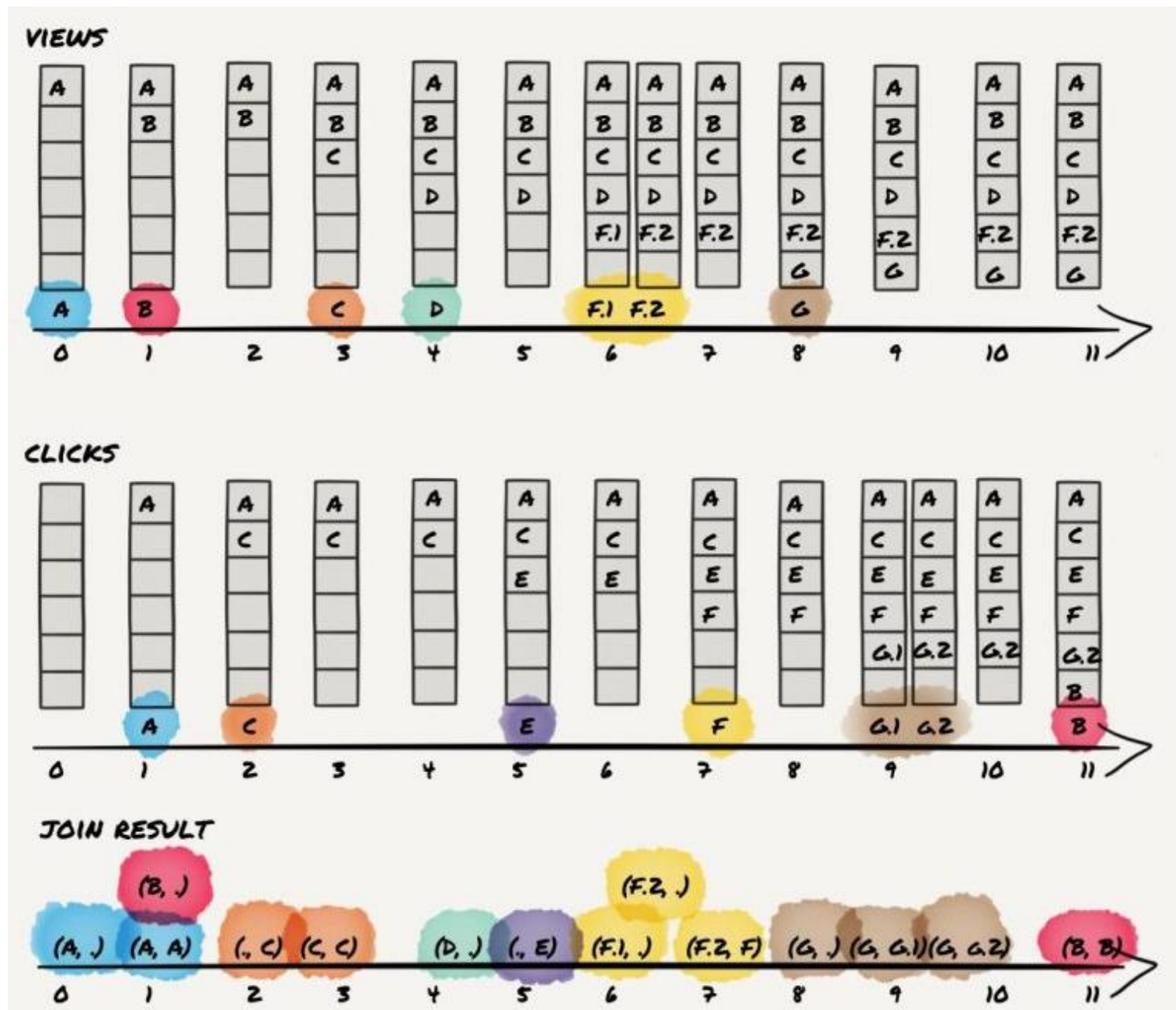




Table to Table Full Outer Join

- The result is the same as with the left join plus the “right join” result records for clicks C and D with an empty left-hand side.
- This join is pretty close to SQL semantics and, thus, easy to understand.
- The difference to plain SQL is that the result automatically updates when new data arrives on the streams. Thus, the result can be described as an ever-updating view of the table join.



POLITECNICO
MILANO 1863

Stream to Table Joins



Stream to Table joins require the `unidirectional` keyword

- The `unidirectional` keyword can be used in the from clause to identify streams that provide the events to execute the join.
- If the keyword is present for a stream, all other streams in the from clause become passive streams. When events arrive or leave a data window of a passive stream, then the join does not generate join results.
- Therefore, the `unidirectional` keyword makes the stream-table join asymmetric: only the (left) stream input triggers a join computation. Because the join is not-windowed, the (left) input stream is stateless; thus, join lookups from table records to stream records are impossible.
- This semantics is usually used to enrich a data stream with auxiliary information from a table. However, for the example in use, we'll use the "views" as the left stream and "clicks" as the right table input.



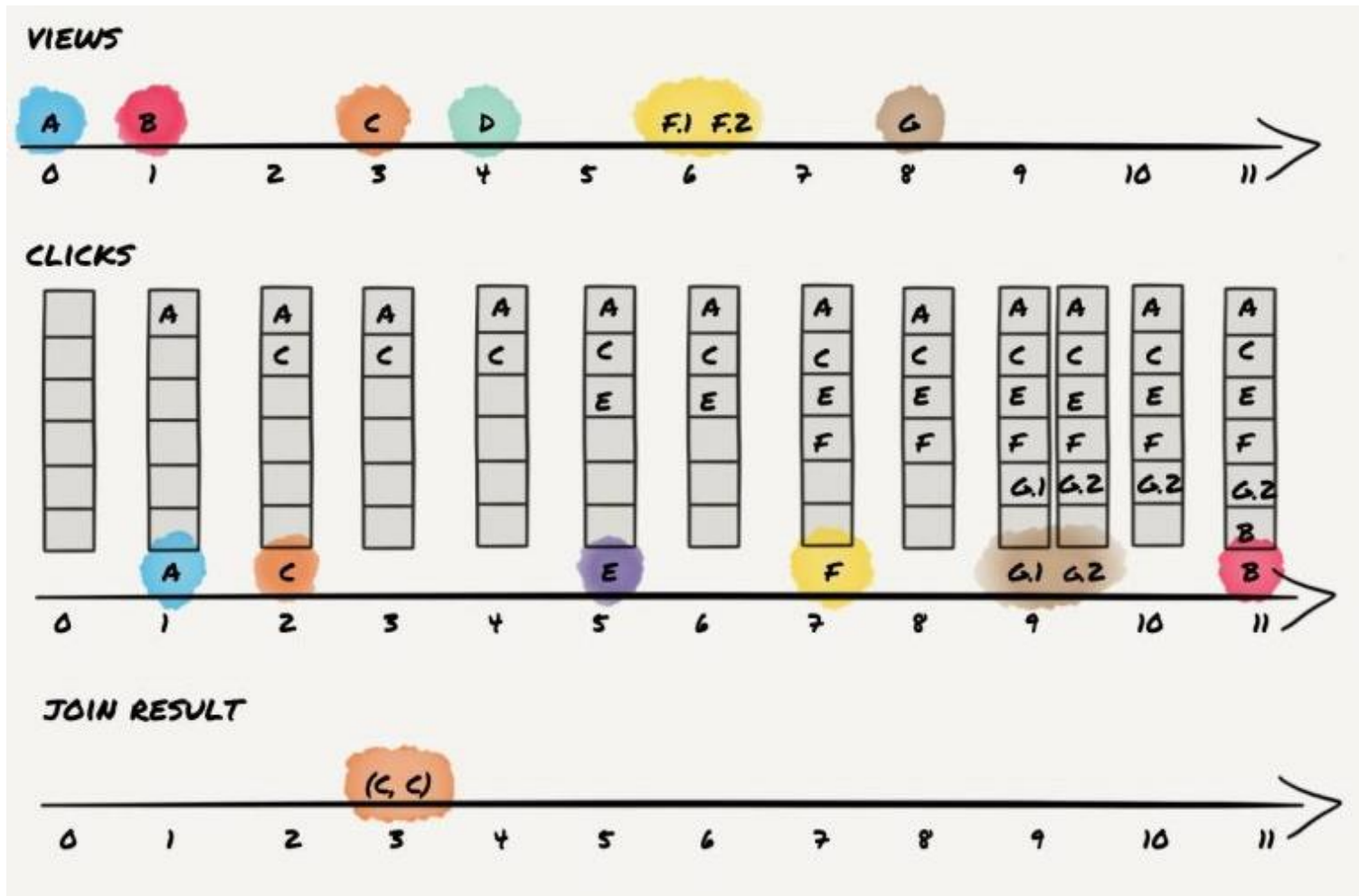
Stream to Table Inner Join

- Query

```
select *  
from View as v  
    unidirectional inner join  
    Click#unique(id) as c  
on v.id = c.id;
```



Stream to Table Inner Join





Stream to Table Inner Join

- The result is just a single record, as click C is the only click that arrives before the corresponding view event.



Stream to Table

Left Outer Join

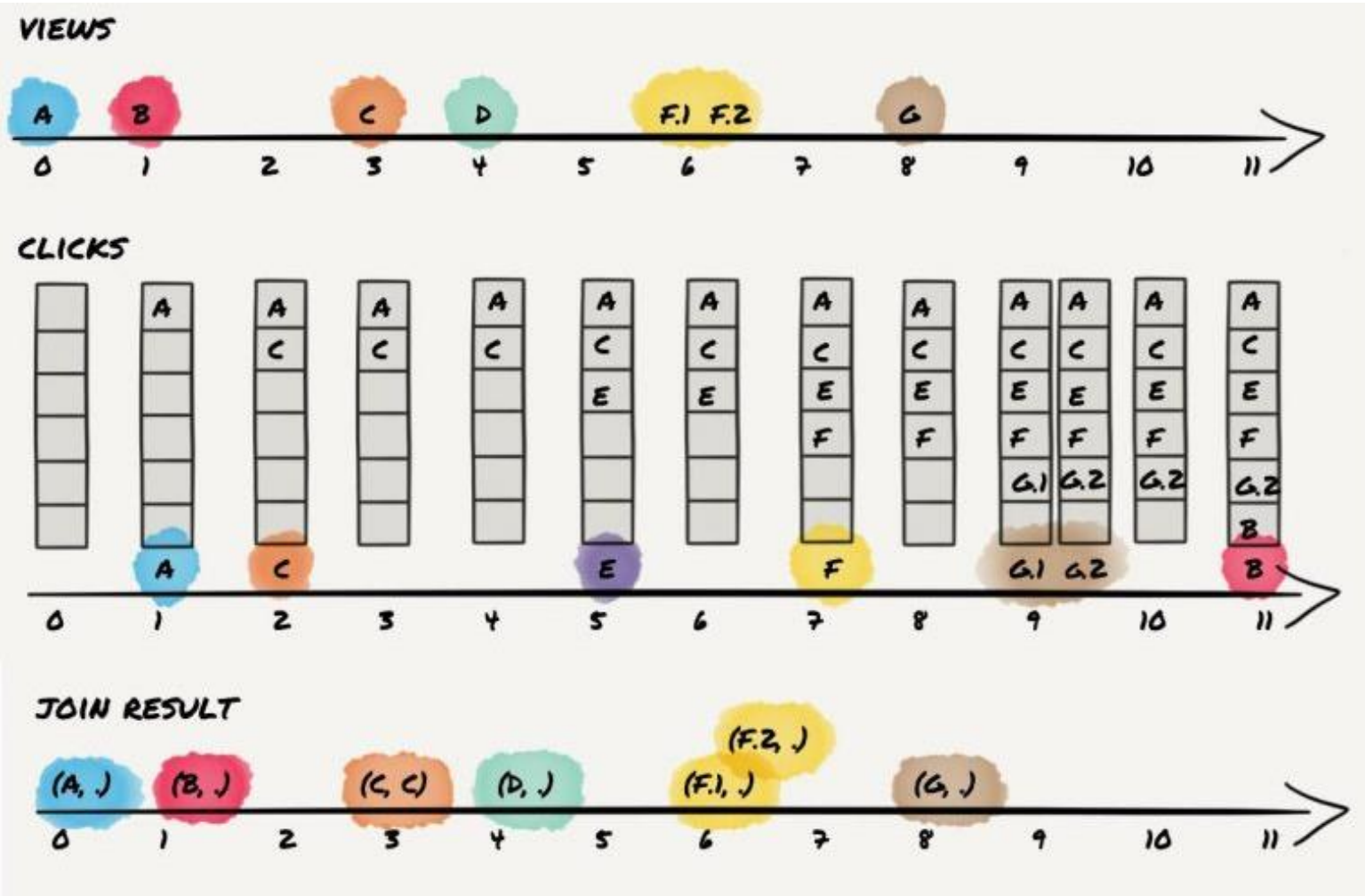
- Query

```
select *  
from View as v  
    unidirectional left outer join  
    Click#unique(id) as c  
on v.id = c.id;
```



Stream to Table

Left Outer Join





Stream to Table Left Outer Join

- As expected, we get the inner C/C join result as well as one join result for each (left) stream record



License and Acknowledgment

- This work is licensed under the Creative Commons Attribution-ShareAlike International Public License
- It is largely based on “Crossing the Streams – Joins in Apache Kafka” by Florian Troßbach, written in May 2017 on codecentric and republished in September 2017 on Confluent’s blog

<https://www.codecentric.de/wissens-hub/blog/crossing-streams-joins-apache-kafka>

<https://www.confluent.io/blog/crossing-streams-joins-apache-kafka/>

EPL and Esper

Streaming joins and more

Emanuele Della Valle

prof @ Politecnico di Milano

founder @ Quantia Consulting

founder & CRO @ motus ml



POLITECNICO
MILANO 1863